

AURORA: Active Uncertainty-Driven Re-Orientation for In-Hand Reconstruction

Anonymous Author(s)

Affiliation

Address

email

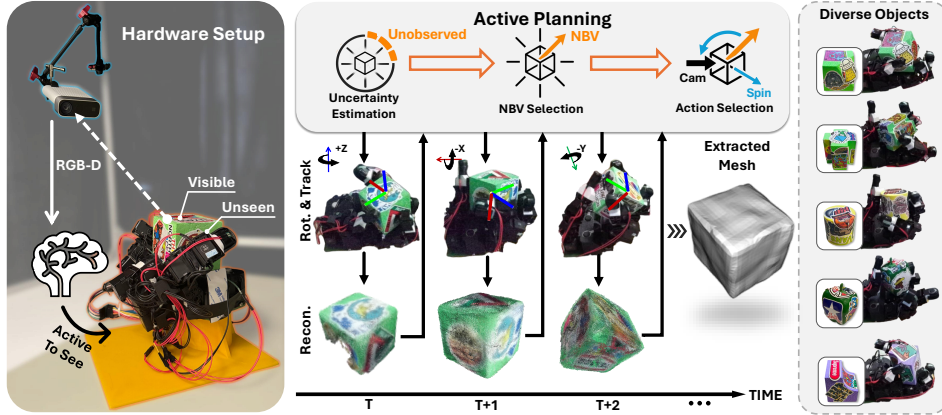


Figure 1: AURORA, a framework for active in-hand object reconstruction using a fixed RGB-D camera. The hand reorients an object using an in-hand manipulation policy and an uncertainty-driven next-best-view planner, exposing unobserved regions and accelerating 3D reconstruction.

Abstract: Observing objects grasped by a robot hand is challenging due to severe visual occlusions. The manipulator itself obscures large portions of the object, while a fixed camera provides viewpoint-constrained observations. Although these hidden surfaces can be exposed by in-hand manipulation, most existing approaches rely on predefined or open-loop reorientation strategies that do not explicitly target unobserved regions. To address this limitation, we propose AURORA, an active 3D reconstruction framework that leverages in-hand manipulation for object inspection. At the core of AURORA is Ray-GPIS, a novel informative planner that estimates reconstruction uncertainty along candidate viewing rays. By identifying directions that maximize uncertainty-novelty score, Ray-GPIS generates next-best-view targets, which are executed via an axis-conditioned in-hand rotation policy. The resulting visual observations are integrated through a visual fusion pipeline for incremental 3D reconstruction. Extensive real-world experiments on objects with diverse geometries demonstrate that AURORA outperforms baselines, achieving higher surface completeness and faster reconstruction. The project webpage is available at <https://aurorahand.github.io/>

Keywords: Probabilistic learning and uncertainty in robotics, Multimodal perception, sensor fusion, and robot vision, Active In-Hand Object Reconstruction

1 Introduction

Human perception is inherently active: when inspecting an object, humans deliberately reorient it to reveal surfaces occluded by the hand [1]. Robotic systems have a similar need during manipulation, where hand-object occlusions and limited camera viewpoints lead to partial observability. Unlike humans, who can simultaneously coordinate vision and manipulation to acquire novel views, most

robotic systems rarely leverage their physical dexterity to resolve such occlusions, restricting the information available for understanding the object.

A primary reason for this limitation is the lack of a planning framework that translates perceptual needs into manipulation actions. While prior research has explored using robot manipulation data for object understanding, these approaches typically reconstruct objects from passively collected manipulation sequences, such as pick-and-place interactions [2] or in-hand rotation videos [3]. In these methods, the robot follows predefined action sequences or open-loop policies, serving merely as a passive data collector without determining where to observe next. Because they do not explicitly reason about viewpoint informativeness or convert perceptual uncertainty into action decisions, such pipelines are time-inefficient and may leave occluded regions persistently under-observed.

To efficiently and accurately recover the complete 3D geometry of unknown objects, we propose an active in-hand reconstruction framework that tightly couples visual perception with manipulation in a closed-loop manner. Rather than executing predefined motion sequences, the system follows an informative policy that actively selects actions to improve perceptual coverage. At each time step, the robot explicitly represents spatial geometric uncertainty over the object surface, which is then used to plan the next in-hand reorientation action, accelerating the exposure of under-observed regions. On the basis of the collected views, we implement a geometric fusion framework to integrate multi-view observations. Together, these components enable fully autonomous object reconstruction through coordinated hand–eye interactions.

In summary, the contributions of this paper include:

- AURORA, an informative framework for capturing complete object geometry in-hand, enabling autonomous exploration with improved efficiency and completeness.
- Ray-GPIS, a ray-conditioned planner that converts GPIS point-wise uncertainty into direction-wise view scores for active in-hand reorientation.
- An online object-centric fusion pipeline, and extensive real-world evaluations showing improved completeness and exploration efficiency under hand-object occlusion.

2 Related Works

2.1 In-Hand Manipulation for Object Reconstruction

In-hand manipulation enables a robot to reorient an object within its grasp, providing a natural mechanism for exposing previously unseen surfaces. Existing approaches have studied both model-based manipulation, which plans motions using contact mechanics [4, 5], and learning-based manipulation, which improves robustness to novel objects and complex dynamics [6, 7, 8, 9, 10]. Recent works further incorporate tactile sensing to improve contact perception and dexterous control [11, 12].

However, most in-hand manipulation methods optimize for reorientation stability, speed, or task success, rather than reconstruction quality. A recent work NeuralFeels [3] is closely related, as it reconstructs in-hand object geometry from visuotactile observations. Despite this similarity, NeuralFeels reconstructs object pose and shape from a given sequence of visual and high-resolution tactile interactions, but does not include an active planning mechanism. Although a direct benchmark is not feasible due to differences in hardware settings, we discuss the relationship between NeuralFeels and our work along with limited comparisons in Appendix F.4.

2.2 Object Reconstruction and Tracking under Occlusion

Object reconstruction from image sequences has been widely studied, with approaches ranging from Structure-from-Motion [13] and RGB-D fusion to neural implicit representations [14, 15, 16, 17]. These methods usually assume static scene geometry and establish multi-view consistency primarily through camera motion. In contrast, in robotic in-hand object reconstruction, the object moves unpredictably under contact and is occluded by the hand, making such multi-view consistency more

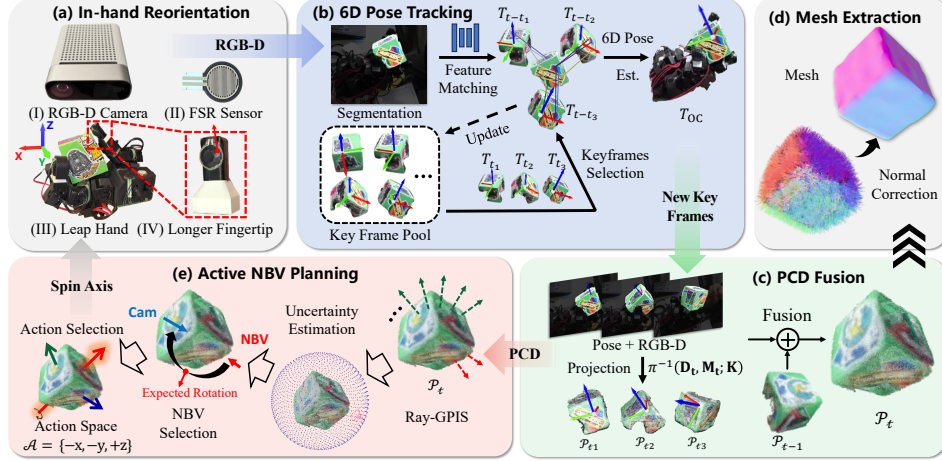


Figure 2: Overview of our technical pipeline. The system integrates four modules: (a) in-hand object reorientation with the Leap Hand; (b) 6D pose tracking via BundleTrack [18]; (c–d) reconstructions; and (e) uncertainty-driven next-best-view planning.

difficult to achieve. Recent model-free trackers and foundation-model-based pose estimators provide useful tools for addressing this issue [18, 19, 20, 21], and have been used to address similar occlusion challenges arising in human–object interaction reconstruction [22, 23, 24, 25]. However, these works primarily reconstruct objects from observed sequences and do not address active action or viewpoint selection for revealing previously unobserved object regions.

2.3 Active Exploration and Uncertainty-Driven View Planning

Active perception seeks to plan sensing actions that maximize task-relevant information acquisition. Most existing systems explore static, unknown environments using mobile sensors [26, 27, 28]. These methods use sensing modalities such as cameras or tactile sensors [29] and typically optimize objectives that reduce geometric uncertainty or maximize expected information gain [30]. Common formulations include frontier-based exploration of unknown space [31], as well as information-theoretic criteria based on Fisher information [32], mutual information [33], and related measures. Beyond mobile-camera exploration, recent work has also studied active object reconstruction with fixed sensors and manipulable objects. For instance, a robot can grasp and re-pose an object in front of a static RGB-D camera to expose previously occluded surfaces [34]. Other interactive perception approaches use purposeful manipulation to incrementally reveal unseen geometry and improve reconstruction completeness [35]. However, these methods largely assume controlled object reposing or external manipulation, rather than continuous in-hand manipulation. As a result, they do not address the coupled challenges of persistent hand-induced occlusion, object motion during interaction, and action selection for revealing uncertain object regions in the robot hand.

3 Methodology

The overall pipeline of AURORA is illustrated in Fig. 2. We tackle in-hand 3D reconstruction as a closed-loop perception–action problem. From an initial observation, we iterate: (i) select a rotation axis and execute an in-hand rotation; (ii) track the object pose and select keyframes; (iii) fuse keyframes into an incremental point-cloud reconstruction; and (iv) update an uncertainty model and compute the next-best view (NBV) for the next action. When the loop terminates, we extract a watertight mesh as the final reconstruction.

3.1 In-Hand Object Reorientation

We use a tactile-based in-hand rotation policy adapted from Rotating without Seeing [11] as the low-level action executor. Given a commanded rotation axis, the policy reorients the grasped object

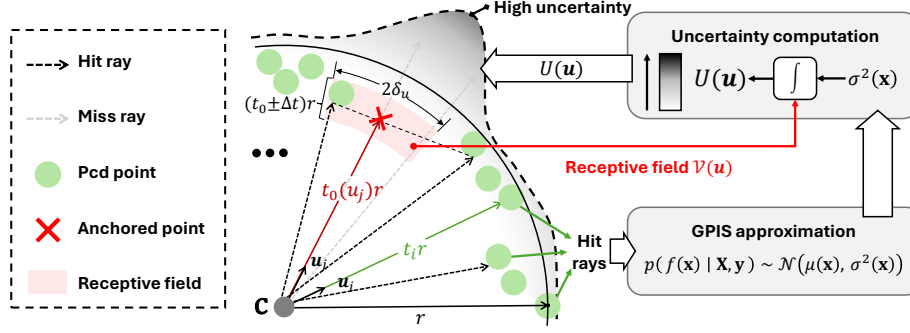


Figure 3: Ray-GPIS estimates direction-wise reconstruction uncertainty. Rays are cast from the object center $\mathbf{c}$. Each direction $\mathbf{u}$ is assigned an anchor point $\mathbf{x}_0(\mathbf{u})$ near the predicted surface intersection, and GPIS variance is aggregated in a local receptive field to obtain $U(\mathbf{u})$.

101 accordingly. We deploy the policy on a Leap Hand [36]; hardware modifications, tactile sensing,
 102 and policy training details are provided in Appendix A.

103 We define our action space in a hand-centered world frame W , whose origin is at the palm center.
 104 The z -axis aligns with the outward palm normal, and the x - and y -axes lie in the palm plane. The
 105 feasible action set is:

$$\mathcal{A} \triangleq \{-x, -y, +z\}, \quad (1)$$

106 where each element denotes a rotation-axis primitive in W . At each planning step, the active planner
 107 selects $\mathbf{a}^* \in \mathcal{A}$, which is then executed by the low-level policy.

108 3.2 Ray-GPIS Active View Planning

109 Actions are selected based on reconstruction uncertainty over object geometry. While existing GPIS-
 110 based methods [37, 29] estimate point-wise uncertainty in 3D space, they do not directly provide
 111 view-dependent scores for in-hand exploration. To address this, Ray-GPIS converts point-wise spa-
 112 tial GPIS uncertainty into direction-wise planning scores by casting object-centered rays, anchoring
 113 them near predicted surface locations, and aggregating posterior variance within local receptive
 114 fields, as shown in Fig. 3.

115 3.2.1 Ray-GPIS Uncertainty Estimation

116 Specifically, we use the center $\mathbf{c}$ of the oriented bounding box of $\mathcal{P}_t$ as the ray origin and uniformly
 117 sample N unit directions $\{\mathbf{u}_i\}_{i=1}^N$ on the sphere. Each ray is parameterized by a normalized radial
 118 coordinate t_i :

$$\mathbf{x}(t_i; \mathbf{u}_i) = \mathbf{c} + t_i r \mathbf{u}_i, \quad t_i \in [0, t_{\max}], \quad (2)$$

119 where r is the maximum distance from $\mathbf{c}$ to any point in $\mathcal{P}_t$, and t_i denotes the radial distance
 120 normalized by r . Thus, $t_i = 1$ approximately corresponds to the current point-cloud extent along
 121 the object scale, while $t_{\max} > 1$ allows the ray to probe slightly beyond the observed geometry.

122 For each direction, Ray-GPIS assigns an anchor point $\mathbf{x}_0(\mathbf{u}_i) = \mathbf{x}(t_0(\mathbf{u}_i); \mathbf{u}_i)$, which approximates
 123 the surface location along the ray. A ray is classified as a *hit ray* if it intersects the observed surface
 124 neighborhood represented by $\mathcal{P}_t$; its anchor is placed at the first observed surface intersection. A
 125 *miss ray* has no reliable observed intersection. For miss rays, the anchor depth is inferred by inter-
 126 polating nearby hit-ray depths on the viewing sphere. This enables Ray-GPIS to query uncertainty
 127 both around observed surfaces and near plausible surface locations in unobserved directions. Details
 128 of the definitions and implementation are provided in Appendix B.1.

129 We then fit a GPIS [37] model to the observed anchors and use its posterior variance $\sigma^2(\mathbf{x})$ as
 130 the spatial reconstruction uncertainty. The GPIS is trained only on hit-ray anchors, while miss-ray
 131 anchors are used only as query locations for uncertainty evaluation. To obtain a direction-wise score,

we evaluate the variance within a local receptive field around each anchor:

$$\mathcal{V}(\mathbf{u}_i) = \{\mathbf{x}(t_i; \mathbf{u}) \mid t_i \in [t_0(\mathbf{u}_i) - \Delta t, t_0(\mathbf{u}_i) + \Delta t], \angle(\mathbf{u}, \mathbf{u}_i) \leq \delta_u\}. \quad (3)$$

Here, Δt defines a narrow radial band around the anchor depth, and δ_u defines the angular neighborhood on the viewing sphere.

The direction-wise uncertainty is computed by integrating the GPIS variance over the receptive field:

$$U(\mathbf{u}_i) = \int_{\mathcal{V}(\mathbf{u}_i)} \sigma^2(\mathbf{x}) d\mathbf{x}, \quad (4)$$

Details of GPIS training and posterior inference are provided in Appendix B.2.

3.2.2 Next-Best-View Selection

Given the direction-wise uncertainty $U(\mathbf{u}_i)$ estimated by Ray-GPIS, the next view should target regions that are both uncertain and insufficiently observed. However, directly maximizing $U(\mathbf{u}_i)$ may select directions close to the current fused geometry, leading to redundant observations. We therefore introduce a novelty weight based on the distance between each candidate receptive field and the current point cloud $\mathcal{P}_t$. For a candidate direction $\mathbf{u}_i$, let $\bar{d}_{\text{nn}}(\mathbf{u}_i)$ be the average nearest-neighbor distance from samples in $\mathcal{V}(\mathbf{u}_i)$ to $\mathcal{P}_t$. We normalize it by the median 1-NN spacing $\bar{d}_{1\text{nn}}$ of $\mathcal{P}_t$, which represents the typical point-cloud resolution, and define

$$\eta(\mathbf{u}_i) = 1 - \exp\left(-\left(\frac{\bar{d}_{\text{nn}}(\mathbf{u}_i)}{\beta \bar{d}_{1\text{nn}}}\right)^2\right), \quad (5)$$

where β controls the scale at which a direction is considered novel. The final NBV direction is selected by maximizing the product of reconstruction uncertainty and geometric novelty:

$$\mathbf{u}_{\text{NBV}}^* = \arg \max_{\mathbf{u}_i} U(\mathbf{u}_i) \eta(\mathbf{u}_i). \quad (6)$$

This encourages the planner to prioritize regions that are not only uncertain, but also far from already fused observations.

3.2.3 Action Selection

The selected NBV direction specifies which object-side region should be exposed to the fixed camera, but it must still be converted into an executable in-hand rotation primitive. Since the low-level controller operates in the hand-centered world frame W , we first transform the object-frame NBV direction into W :

$$\mathbf{d}_W = \mathbf{R}_{WO}(t) \mathbf{u}_{\text{NBV}}^*, \quad (7)$$

where $\mathbf{R}_{WO}(t)$ is the rotation component of $\mathbf{T}_{WO}(t) = \mathbf{T}_{WC} \mathbf{T}_{CO}(t)$. We define $\mathbf{v}_W$ as the unit vector from the current object center to the camera, expressed in W . We compute the minimal object rotation that aligns the desired direction $\mathbf{d}_W$ with $\mathbf{v}_W$, and represent it by the Lie-algebra vector $\boldsymbol{\omega} \in \mathbb{R}^3$. Since the controller only supports a discrete action set $\mathcal{A}$, we choose the primitive whose axis is most aligned with this desired rotation:

$$\mathbf{a}^* = \arg \max_{\mathbf{a} \in \mathcal{A}} \mathbf{a}^\top \boldsymbol{\omega}. \quad (8)$$

Details of the frame transformation and rotation-vector computation are provided in Appendix C.

3.3 Object-Centric Tracking and Reconstruction

Object-centric fusion and Ray-GPIS planning require object poses in RGB-D frames. We use BundleTrack [18], a CAD-free RGB-D tracker, to estimate $T_{CO}(t) \in SE(3)$ from segmented observations [38] and fuse them in a consistent object frame. To reduce the effect of hand-object occlusions, we apply a lightweight visibility-aware keyframe filter that retains only frames with sufficient object visibility, stable pose estimates, and adequate motion from the latest keyframe. Detailed tracking and keyframe-selection rules are provided in Appendix D.

Given the selected keyframes $\mathcal{K}$, we maintain a compact object-centric point cloud as the geometric state for closed-loop planning. For each keyframe $t \in \mathcal{K}$, masked depth pixels are back-projected into the camera frame, transformed into the object frame, and fused incrementally:

$$\mathcal{P}_t \triangleq \mathcal{C}(\mathcal{P}_{t-1} \cup T_{CO}(t)^{-1}\pi^{-1}(\mathbf{D}_t, \mathbf{M}_t; \mathbf{K}_{\text{cam}})), \quad (9)$$

where $\mathbf{D}_t$, $\mathbf{M}_t$, and $\mathbf{K}_{\text{cam}}$ denote the depth image, object mask, and camera intrinsics, respectively; $\pi^{-1}(\cdot)$ back-projects masked depth pixels into 3D camera-frame points; and $\mathcal{C}(\cdot)$ applies voxel downsampling and outlier removal. The resulting point cloud $\mathcal{P}_t$ is used by Ray-GPIS for uncertainty estimation and NBV planning.

After exploration, we convert the final fused point cloud $\mathcal{P}_T$ into a watertight mesh for quantitative evaluation. We estimate globally consistent normals using FaCE [39] and reconstruct the mesh with NKSr [40]; details are provided in Appendix E.

4 Experiments

We evaluate the proposed active in-hand reconstruction framework on six graspable real-world objects with varied geometries. Section 4.1 describes the hardware setup and experimental protocol. Section 4.2 evaluates reconstruction accuracy, while Sec. 4.3 analyzes reconstruction efficiency compared with non-active manipulation baselines.

4.1 System Setup and Task Protocol

Our system uses a fixed Azure Kinect DK RGB-D camera for visual sensing and a Leap Hand [36] for in-hand manipulation. During execution, the hand rotates the object for 6 s; the planner then updates the reconstruction and replans the next rotation direction. This closed-loop process is repeated under a fixed 30 s interaction budget. Then, we evaluate both the online point-cloud reconstruction used for active planning and the final mesh extracted after exploration. For quantitative evaluation, we obtain ground-truth meshes using an EinScan Pro 2X high-resolution 3D scanner. More detailed hardware specifications and hyperparameters for the planner and reconstruction are provided in Appendix A.1 and F.1, respectively.

4.2 Reconstruction Accuracy

The first evaluation focuses on the reconstruction quality of the proposed AURORA framework. We evaluate reconstruction performance on six graspable real-world objects. For each, we report both the online reconstructed point cloud and the offline refined mesh obtained under a fixed manipulation budget of 30 s. Qualitative and quantitative results are shown in Fig. 4 and Tab. 1, respectively.

Reconstruction accuracy is evaluated using the F-score metric, where $F@ \tau$ denotes the harmonic mean of precision and recall within a distance tolerance τ (higher is better). As summarized in Tab. 1(a), AURORA achieves strong reconstruction performance under a fixed 30 s budget. The online point-cloud reconstruction already reaches high accuracy, with an average $F@10$ of 0.9671 and $F@5$ of 0.8249, showing that active reorientation can efficiently acquire informative observations. Offline mesh refinement further improves the reconstruction quality, increasing the mesh-level $F@5$ to 0.8624 while preserving a high $F@10$ of 0.9672. As shown in the Tab. 1(b), AURORA also consistently outperforms non-active rotation baselines, including single-axis rotations along the x -, y -, and z -axes and a predefined fixed rotation schedule. This is further supported by the qualitative temporal comparison in Fig. 5(a), where AURORA reconstructs more complete geometry with fewer missing regions and faster surface coverage. These results indicate that uncertainty-driven active reorientation improves both reconstruction efficiency and final surface completeness. More detailed qualitative comparisons with the x -, y -, and z -axis baselines are provided in Appendix F.2.

We further compare our method with recent single-view 3D reconstruction methods, TRELIS.2 [41] and SPAR3D [42], as reference baselines in Tab. 1(b). Although not designed for active in-hand reconstruction, these methods provide a useful reference for reconstruction from incomplete

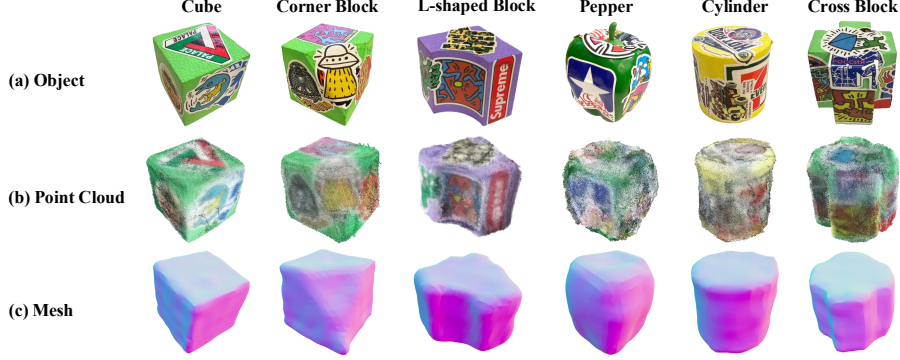


Figure 4: Qualitative results on real-world objects. (a) Test objects. (b) Online point clouds. (c) Offline extracted meshes.

Table 1: Quantitative results after a fixed 30 s budget. $F@τ$ is the harmonic mean of precision and recall at tolerance $τ$. Single-view meshes are evaluated only with Mesh–Mesh scores after post-hoc scale alignment to the ground-truth meshes.

(a) Results of our active strategy across objects.							(b) Mean comparison with baselines.						
Obj.	PCD–PCD (Online)			Mesh–Mesh (Offline)			Method	Mean PCD–PCD			Mean Mesh–Mesh		
	$F@2\uparrow$	$F@5\uparrow$	$F@10\uparrow$	$F@2\uparrow$	$F@5\uparrow$	$F@10\uparrow$		$F@2\uparrow$	$F@5\uparrow$	$F@10\uparrow$	$F@2\uparrow$	$F@5\uparrow$	$F@10\uparrow$
Cube	0.2895	0.9337	0.9977	0.6481	0.9557	0.9957	x -axis	0.1735	0.6625	0.8655	0.3595	0.6211	0.8440
Corner Block	0.2353	0.7354	0.9303	0.5298	0.8559	0.9488	y -axis	0.1463	0.6074	0.7933	0.3082	0.5643	0.7498
L-shaped Block	0.2306	0.8366	0.9886	0.4674	0.8367	0.9450	z -axis	0.2079	0.7786	0.9228	0.4414	0.7231	0.8872
Pepper	0.1770	0.8321	0.9773	0.5480	0.8890	0.9913	Fixed schedule	0.1550	0.7015	0.8995	0.4544	0.6948	0.8904
Cylinder	0.4428	0.8400	0.9425	0.4035	0.8320	0.9372	TRELLIS.2 [41]	–	–	–	0.6538	0.8315	0.9017
Cross Block	0.1454	0.7718	0.9664	0.4601	0.8050	0.9850	SPAR3D [42]	–	–	–	0.5571	0.7386	0.9483
Mean	0.2534	0.8249	0.9671	0.5095	0.8624	0.9672	Ours (Active)	0.2534	0.8249	0.9671	0.5095	0.8624	0.9672
Std.	0.1054	0.0679	0.0263	0.0855	0.0535	0.0262							

visual observations. For a well-defined comparison, we use the corresponding clean object image in Fig. 4(a) as input for each object. Because single-view methods do not recover metric scale, their raw outputs can have arbitrary sizes; we therefore align their mesh to the ground truth scale before computing F-scores. Thus, these scores reflect scale-normalized shape similarity, not directly usable metric reconstructions accuracy. Even after removing scale errors in their favor, our method still achieves higher mean F-scores at the 5 mm and 10 mm thresholds. Moreover, these baselines require clean, high-resolution inputs and may produce incomplete or non-watertight meshes under hand occlusion, limiting their suitability for metric-aware robotic applications. Detailed comparisons and qualitative results are provided in Appendix F.3.

4.3 Efficiency vs. Non-active Baselines

Analysis of Active Reconstruction Efficiency. Second, we evaluate online reconstruction efficiency by tracking the temporal evolution of both reconstruction uncertainty and geometric accuracy under the proposed closed-loop active strategy. We use q_{95} , the 95th percentile of the estimated uncertainty distribution, to indicate poorly observed, high-uncertainty regions (e.g., holes in the reconstruction). We also report the Mesh–Mesh F-score at 5 mm, denoted as $F@5$, to measure the quality of the intermediate reconstructed mesh against the scanned ground truth. As shown by the red curves in Fig. 5(b), our method reaches lower q_{95} and higher $F@5$ over time, indicating effective information acquisition and improved geometric reconstruction quality. The solid red q_{95} curve decreases stepwise: when exploration begins to plateau, each replanning step (every 6 s, marked by vertical lines) introduces additional uncertainty reduction. This suggests that active planning continues to provide useful information gain. The increasing $F@5$ curve further verifies, through an external geometric metric, that the reduced uncertainty translates into better reconstruction. After the final replanning step at 24 s, the curves gradually stabilize, suggesting that the most informative viewpoints have been sufficiently covered and additional observations yield diminishing returns.

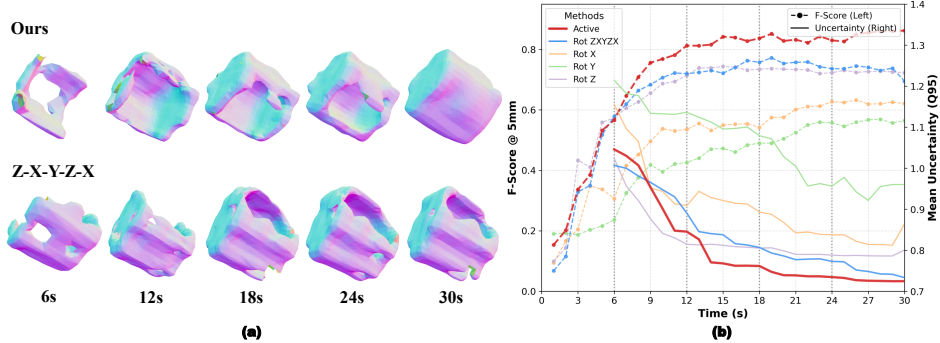


Figure 5: (a) Qualitative comparison of the reconstruction result on *Cross Block* over time between our method and the Z-X-Y-Z-X baseline. (b) Online reconstruction efficiency under different rotation strategies, averaged over six objects. The curves show the q_{95} uncertainty and Mesh-Mesh F-score at the 5mm threshold within a 30s budget.

Comparison of Efficiency over Open-loop Baselines. We further compare our method against open-loop rotation schedules under the same fixed 30s manipulation budget. The dashed $F@5$ curves in Fig. 5(b) show that our method improves reconstruction quality more efficiently. In particular, after the first replanning step at 6s, our $F@5$ curve maintains a faster growth than the baselines, indicating that the selected active rotation provides more informative observations. Single-axis rotations perform worst, with slow F-score improvement and high remaining uncertainty, because they fail to expose surfaces outside their limited reachable set. The predefined schedule performs in between but remains suboptimal. Minor non-monotonic fluctuations in the curves are caused by the automatic fusion and mesh extraction pipeline, where denoising, stochastic point-cloud downsampling, and meshing can slightly change the reconstructed surface over time.

5 Limitations

Our framework has several limitations. First, reconstruction quality depends on accurate 6D pose tracking. For visually featureless or textureless objects, pose drift can introduce fusion artifacts and keyframe misalignment. Incorporating contact or tactile constraints into pose tracking could potentially improve robustness under weak visual texture and severe hand-object occlusion. Second, the replanning interval is fixed at 6s, chosen empirically based on the object rotation speed. An adaptive replanning interval based on uncertainty reduction, viewpoint novelty, or tracking confidence could potentially further improve exploration efficiency. Finally, in-hand manipulation may occasionally cause object slip or drops. This could be mitigated through more robust control policies, and improved sim-to-real transfer. Future work will focus on addressing these limitations.

6 Conclusion

This paper presents AURORA, an active in-hand reconstruction framework that closes the loop between online object-centric reconstruction and uncertainty-driven reorientation. At the core of AURORA is Ray-GPIS, a viewing-ray-conditioned uncertainty estimation module that estimates direction-wise view uncertainty. This formulation facilitates next-best-view planning under severe hand-object occlusion. Together with CAD-free 6D pose tracking and lightweight RGB-D fusion, AURORA actively selects in-hand rotation actions to expose under-observed object regions and progressively improve reconstruction completeness. Real-world experiments show that AURORA achieves more complete reconstructions and faster uncertainty reduction than non-active rotation baselines under the same interaction budget. Compared with recent single-view 3D reconstruction methods, AURORA also achieves higher fidelity by actively acquiring additional views rather than relying solely on learned shape priors. Future work will focus on improving the robustness of object pose tracking and enhancing the stability of in-hand manipulation.

References

- [1] R. Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988.
- [2] M. Krainin, P. Henry, X. Ren, and D. Fox. Manipulator and object tracking for in-hand 3d object modeling. *The International Journal of Robotics Research*, 30(11):1311–1327, 2011.
- [3] S. Suresh, H. Qi, T. Wu, T. Fan, L. Pineda, M. Lambeta, J. Malik, M. Kalakrishnan, R. Calandra, M. Kaess, et al. Neuralfeels with neural fields: Visuotactile perception for in-hand manipulation. *Science Robotics*, 9(96):eadl0628, 2024.
- [4] N. Chavan-Dafle, R. Holladay, and A. Rodriguez. In-hand manipulation via motion cones. *arXiv preprint arXiv:1810.00219*, 2018.
- [5] B. Liang, K. Ota, M. Tomizuka, and D. K. Jha. Robust in-hand manipulation with extrinsic contacts. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6544–6550. IEEE, 2024.
- [6] H. Qi, A. Kumar, R. Calandra, Y. Ma, and J. Malik. In-hand object rotation via rapid motor adaptation. In *Conference on Robot Learning*, pages 1722–1732. PMLR, 2023.
- [7] G. Khandate, S. Shang, E. T. Chang, T. L. Saidi, Y. Liu, S. M. Dennis, J. Adams, and M. Ciocarlie. Sampling-based exploration for reinforcement learning of dexterous manipulation. *arXiv preprint arXiv:2303.03486*, 2023.
- [8] O. M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.
- [9] T. Chen, M. Tippur, S. Wu, V. Kumar, E. Adelson, and P. Agrawal. Visual dexterity: In-hand reorientation of novel and complex object shapes. *Science Robotics*, 8(84):eadc9244, 2023.
- [10] Y. Qin, B. Huang, Z.-H. Yin, H. Su, and X. Wang. Dexpoint: Generalizable point cloud reinforcement learning for sim-to-real dexterous manipulation. In *Conference on Robot Learning*, pages 594–605. PMLR, 2023.
- [11] Z.-H. Yin, B. Huang, Y. Qin, Q. Chen, and X. Wang. Rotating without seeing: Towards in-hand dexterity through touch. *arXiv preprint arXiv:2303.10880*, 2023.
- [12] Y. Yuan, H. Che, Y. Qin, B. Huang, Z.-H. Yin, K.-W. Lee, Y. Wu, S.-C. Lim, and X. Wang. Robot synesthesia: In-hand manipulation with visuotactile sensing. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6558–6565. IEEE, 2024.
- [13] J. L. Schönberger and J.-M. Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [14] J. Lei, S. Sridhar, P. Guerrero, M. Sung, N. Mitra, and L. J. Guibas. Pix2surf: Learning parametric 3d surface models of objects from images. In *European Conference on Computer Vision*, pages 121–138. Springer, 2020.
- [15] J. Munkberg, J. Hasselgren, T. Shen, J. Gao, W. Chen, A. Evans, T. Müller, and S. Fidler. Extracting triangular 3d models, materials, and lighting from images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8280–8290, 2022.
- [16] J. Sun, Y. Xie, L. Chen, X. Zhou, and H. Bao. Neuralrecon: Real-time coherent 3d reconstruction from monocular video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15598–15607, 2021.
- [17] D. Azinović, R. Martin-Brualla, D. B. Goldman, M. Nießner, and J. Thies. Neural rgb-d surface reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6290–6301, 2022.

- [18] B. Wen and K. Bekris. Bundletrack: 6d pose tracking for novel objects without instance or category-level 3d models. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8067–8074. IEEE, 2021.
- [19] B. Wen, W. Yang, J. Kautz, and S. Birchfield. Foundationpose: Unified 6d pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024.
- [20] T. Lee, B. Wen, M. Kang, G. Kang, I. S. Kweon, and K.-J. Yoon. Any6d: Model-free 6d pose estimation of novel objects. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 11633–11643, 2025.
- [21] M. Liu, S. Li, A. Chhatkuli, P. Truong, L. Van Gool, and F. Tombari. One2any: One-reference 6d pose estimation for any object. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 6457–6467, 2025.
- [22] S. Jiang, Q. Ye, R. Xie, Y. Huo, and J. Chen. Hand-held object reconstruction from rgb video with dynamic interaction. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12220–12230, 2025.
- [23] Y. Huang, O. Taheri, M. J. Black, and D. Tzionas. Intercap: Joint markerless 3d tracking of humans and objects in interaction. In *DAGM German Conference on Pattern Recognition*, pages 281–299. Springer, 2022.
- [24] Y. Jiang, S. Jiang, G. Sun, Z. Su, K. Guo, M. Wu, J. Yu, and L. Xu. Neuralhofusion: Neural volumetric rendering under human-object interactions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6155–6165, 2022.
- [25] Z. Cao, I. Radosavovic, A. Kanazawa, and J. Malik. Reconstructing hand-object interactions in the wild. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12417–12426, 2021.
- [26] C. Cao, J. Zhang, M. Travers, and H. Choset. Hierarchical coverage path planning in complex 3d environments. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3206–3212. IEEE, 2020.
- [27] C. Feng, H. Li, M. Zhang, X. Chen, B. Zhou, and S. Shen. Fc-planner: A skeleton-guided planning framework for fast aerial coverage of complex 3d scenes. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8686–8692. IEEE, 2024.
- [28] B. Zhou, Y. Zhang, X. Chen, and S. Shen. Fuel: Fast uav exploration using incremental frontier structure and hierarchical planning. *IEEE Robotics and Automation Letters*, 6(2):779–786, 2021.
- [29] F. Zhao and C. Xiao. Autonomous exploration for shape reconstruction and measurement via informative contact-guided planning. *IEEE Robotics and Automation Letters*, 11(2):1250–1257, 2025.
- [30] C. Stachniss, G. Grisetti, and W. Burgard. Information gain-based exploration using rao-blackwellized particle filters. In *Robotics: Science and systems*, volume 2, pages 65–72, 2005.
- [31] B. Yamauchi. A frontier-based approach for autonomous exploration. In *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA’97: Towards New Computational Principles for Robotics and Automation*, pages 146–151. IEEE, 1997.
- [32] W. Jiang, B. Lei, and K. Daniilidis. Fisherrf: Active view selection and mapping with radiance fields using fisher information. In *European Conference on Computer Vision*, pages 422–440. Springer, 2024.

- 358 [33] Y. Xie, Y. Cai, Y. Zhang, L. Yang, and J. Pan. Gauss-mi: Gaussian splatting shannon mutual
359 information for active 3d reconstruction. *arXiv preprint arXiv:2504.21067*, 2025.
- 360 [34] M. Krainin, B. Curless, and D. Fox. Autonomous generation of complete 3d object models
361 using next best view manipulation planning. In *2011 IEEE international conference on robotics
362 and automation*, pages 5031–5037. IEEE, 2011.
- 363 [35] J. Bohg, K. Hausman, B. Sankaran, O. Brock, D. Kragic, S. Schaal, and G. S. Sukhatme. Inter-
364 active perception: Leveraging action in perception and perception in action. *IEEE Transactions
365 on Robotics*, 33(6):1273–1291, 2017.
- 366 [36] K. Shaw, A. Agarwal, and D. Pathak. Leap hand: Low-cost, efficient, and anthropomorphic
367 hand for robot learning. *Robotics: Science and Systems (RSS)*, 2023.
- 368 [37] O. Williams and A. Fitzgibbon. Gaussian process implicit surfaces. In *Gaussian Processes in
369 Practice*, 2006.
- 370 [38] N. Carion and L. G. et al. Sam 3: Segment anything with concepts, 2025.
- 371 [39] D. Scrivener, D. Cui, E. Coldren, S. M. Abulnaga, M. Bessmeltsev, and E. Chien. Faraday
372 cage estimation of normals for point clouds and ribbon sketches. *ACM Trans. Graph.*, 44(4),
373 2025. ISSN 0730-0301. doi:10.1145/3731212.
- 374 [40] J. Huang, Z. Gojcic, M. Atzmon, O. Litany, S. Fidler, and F. Williams. Neural kernel surface
375 reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern
376 Recognition*, pages 4369–4379, 2023.
- 377 [41] J. Xiang, X. Chen, S. Xu, R. Wang, Z. Lv, Y. Deng, H. Zhu, Y. Dong, H. Zhao, N. J. Yuan, and
378 J. Yang. Native and compact structured latents for 3d generation. *Tech report*, 2025.
- 379 [42] Z. Huang, M. Boss, A. Vasishta, J. M. Rehg, and V. Jampani. Spar3d: Stable point-aware
380 reconstruction of 3d objects from single images. 2025.

381 Appendix

382 A Details of In-Hand Reorientation Policy

383 A.1 Hardware Setup

384 This section summarizes the hardware used for policy training, real-world deployment, tactile sens-
385 ing, and ground-truth mesh acquisition. Our hardware setup consists of an Azure Kinect DK RGB-D
386 camera for visual sensing and a Leap Hand [36] for in-hand manipulation. Since the manipulator is
387 equipped with tactile sensors, the tactile sensor configuration is detailed in Sec. A.3. Policy training
388 is conducted on a workstation equipped with four NVIDIA A40 GPUs (Sec. A.2), while real-world
389 deployment runs on a desktop system with an Intel Core i7-13700 CPU, 32 GB of RAM, and an
390 NVIDIA RTX 4090D GPU with 24 GB of memory. For metric computation in the main manuscript,
391 ground-truth object meshes are captured using an EinScan Pro 2X V2 scanner.

392 A.2 Policy Deployment on the Leap Hand

393 First, our system requires object to be actively manipulated. For achieving in-hand manipulation,
394 we use the tactile-based in-hand rotation policy from *Rotating without Seeing* [11] as the low-level
395 reorientation controller and adapt it to the Leap Hand [36]. Since the original policy was designed
396 for a different hand morphology (Allegro Hand) and a different tactile layout, we modify both the
397 simulation environment and the policy interface to match our hardware.

398 Specifically, we replace the original hand model with the Leap Hand model and update the joint
399 limits, actuator parameters, and fingertip geometry. To improve contact coverage during rotation,
400 we extend the distal fingertip links, as shown in Fig. A.1. We also express commanded rotation
401 axes in the hand-centered world frame W , consistent with the main paper. The policy takes an axis
402 command $\mathbf{a} \in \mathcal{A}$ as input and outputs joint-level motor commands to rotate the grasped object about
403 the commanded axis.

404 Before deployment, we retrain the policy in simulation to match the Leap Hand morphology and our
405 tactile sensing setup. The reward is

$$R = w_{\text{rot}} R_{\text{rot}} + w_{\text{contact}} R_{\text{contact}} + w_{\text{stable}} R_{\text{stable}} - w_{\text{act}} R_{\text{act}} - w_{\text{slip}} R_{\text{slip}} - w_{\text{drop}} R_{\text{drop}}. \quad (\text{A.1})$$

406 Here, R_{rot} encourages angular velocity about the commanded axis, R_{contact} encourages sustained
407 tactile contact, and R_{stable} promotes stable object retention. The penalty terms R_{act} , R_{slip} , and
408 R_{drop} discourage excessive actions, object slip, and dropping, respectively. This reward adaptation
409 encourages the policy to produce reliable axis-conditioned rotations rather than merely maximizing
410 rotation speed.

411 A.3 FSR Sensor Layout and Usage

412 To provide tactile feedback needed by in-hand manipulation, we mount force-sensitive resistor
413 (FSR) sensors across the inner surfaces of Leap Hand fingertips, PIP/DIP phalanges, and palm,
414 as illustrated in Fig. A.1. These regions correspond to the primary contact areas during in-hand
415 rotation. FSR signals are asynchronously updated at approximately 50 Hz in a background thread,
416 providing the policy with the most recent contact state at each execution step.

417 The FSR signals are used only by the low-level in-hand reorientation policy. Although they are not
418 used by the reconstruction pipeline, they benefit reconstruction indirectly by enabling more reliable
419 object reorientation.

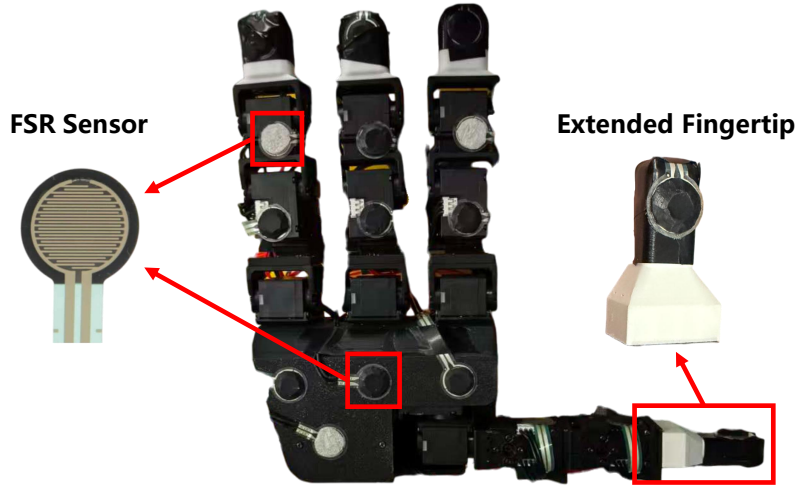


Figure A.1: FSR sensor layout on the Leap Hand. Sensors are embedded across the fingertips, distal and middle phalanges, and the palm to provide tactile feedback for the in-hand rotation policy.

420 A.4 FSR Sensor Activation and Utilization

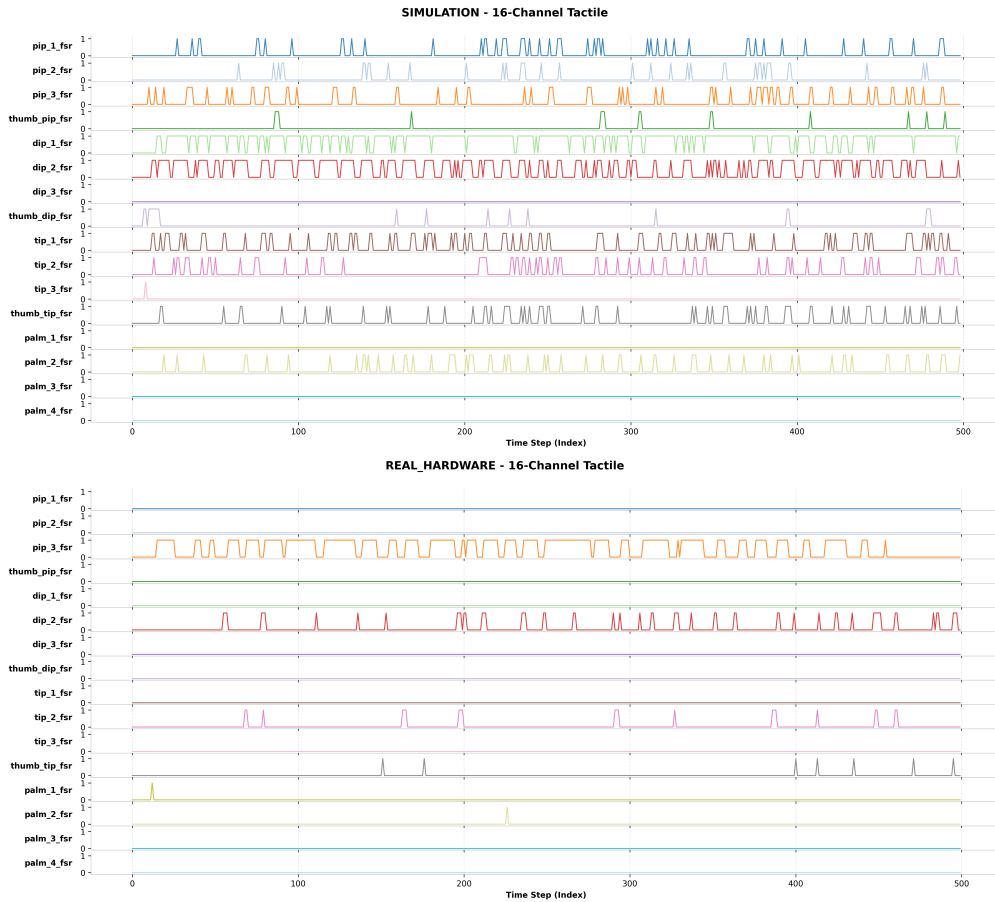


Figure A.2: Temporal activation of the 16-channel FSR tactile signals in simulation and real-hardware deployment.

To better understand how tactile feedback is used by the in-hand rotation policy, we visualize the temporal activations of the 16-channel FSR tactile signals in both simulation and real-hardware deployment. Each curve corresponds to one FSR channel, and a pulse indicates that the corresponding tactile sensor is activated by contact during manipulation.

Fig. A.2 shows that the simulated policy produces rich and distributed tactile activations across multiple fingers and palm sensors, indicating that the learned policy actively exploits contact information from different regions of the hand during object rotation. In real-hardware deployment, the activations become sparser and are concentrated on several dominant contact channels. This discrepancy is mainly caused by the physical characteristics of the real tactile sensors, including the minimum force threshold required for activation, hysteresis during activation and deactivation, and the limited stiffness of the manipulated object, which differs from the simulation setting. Despite these differences, the real system can still reliably obtain consistent tactile feedback from key contact regions, leading to successful manipulation. These observations also suggest that further improvements in sensor placement, calibration, and tactile signal normalization may be a possible way to increase the number of informative channels and improve the stability of real-world in-hand manipulation.

B Details of Ray-GPIS Uncertainty Estimation

This section provides the mathematical details omitted from Sec. 3.2, including hit/miss ray assignment, anchor interpolation, and GPIS posterior variance.

B.1 Hit/Miss Ray Assignment and Anchor Interpolation

Given the current fused point cloud $\mathcal{P}_t$, Ray-GPIS samples a set of candidate viewing directions $\{\mathbf{u}_i\}_{i=1}^N$ and casts rays from the object center $\mathbf{c}$:

$$\mathbf{x}(t; \mathbf{u}_i) = \mathbf{c} + (tr)\mathbf{u}_i, \quad t \in [0, t_{\max}], \quad (\text{A.2})$$

where r normalizes the ray extent. In our implementation, each ray is discretized into a set of depth samples $\{t_{ij}\}_{j=1}^{N_t}$. We compute the minimum distance from each ray to the observed point cloud:

$$d_{\min}(\mathbf{u}_i) = \min_j \min_{\mathbf{p} \in \mathcal{P}_t} \|\mathbf{x}(t_{ij}; \mathbf{u}_i) - \mathbf{p}\|_2. \quad (\text{A.3})$$

A ray is treated as a hit ray if this distance is below a threshold τ_{hit} :

$$m_i = \mathbb{I}[d_{\min}(\mathbf{u}_i) < \tau_{\text{hit}}]. \quad (\text{A.4})$$

For a hit ray, the anchor depth $t_0(\mathbf{u}_i)$ is defined as the first depth at which the ray enters the observed surface neighborhood:

$$t_0(\mathbf{u}_i) = \min \left\{ t_{ij} \mid \min_{\mathbf{p} \in \mathcal{P}_t} \|\mathbf{x}(t_{ij}; \mathbf{u}_i) - \mathbf{p}\|_2 < \tau_{\text{hit}} \right\}. \quad (\text{A.5})$$

The corresponding anchor point is

$$\mathbf{x}_0(\mathbf{u}_i) = \mathbf{x}(t_0(\mathbf{u}_i); \mathbf{u}_i). \quad (\text{A.6})$$

For a miss ray, no reliable observed surface intersection is available. We therefore infer a plausible anchor depth by interpolating the depths of nearby hit-rays on the viewing sphere. Let $\mathcal{H}_i$ denote the set of nearby hit directions:

$$\mathcal{H}_i = \{j \mid m_j = 1, \mathbf{u}_i^\top \mathbf{u}_j \geq \gamma\}, \quad (\text{A.7})$$

where γ is an angular similarity threshold. We compute cosine-based interpolation weights:

$$\alpha_{ij} = \frac{\exp((\mathbf{u}_i^\top \mathbf{u}_j - 1)/\tau_{\text{interp}})}{\sum_{\ell \in \mathcal{H}_i} \exp((\mathbf{u}_i^\top \mathbf{u}_\ell - 1)/\tau_{\text{interp}})}, \quad j \in \mathcal{H}_i, \quad (\text{A.8})$$

where τ_{interp} controls the angular interpolation bandwidth. The miss-ray anchor depth is then estimated as

$$t_0(\mathbf{u}_i) = \sum_{j \in \mathcal{H}_i} \alpha_{ij} t_0(\mathbf{u}_j). \quad (\text{A.9})$$

If no valid neighboring hit ray exists, the ray is excluded from the current planning update. This anchor interpolation allows Ray-GPIS to evaluate uncertainty near plausible surface locations, even for directions without direct observations.

B.2 GPIS Posterior Variance

Ray-GPIS uses a Gaussian Process (GP) model to estimate spatial reconstruction uncertainty. Given the anchor set

$$\mathbf{X} = \{\mathbf{x}_0(\mathbf{u}_i)\}_{i=1}^{N_x}, \quad (\text{A.10})$$

and the corresponding labels $\mathbf{y}$, we model the implicit function as

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu_0(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')), \quad (\text{A.11})$$

where $\mu_0(\cdot)$ is the prior mean and $k(\cdot, \cdot)$ is the covariance kernel.

For a query point $\mathbf{x}$, the posterior distribution is

$$p(f(\mathbf{x}) \mid \mathbf{X}, \mathbf{y}) \sim \mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x})). \quad (\text{A.12})$$

The predictive mean and variance are

$$\mu(\mathbf{x}) = \mu_0(\mathbf{x}) + \mathbf{k}_x^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} (\mathbf{y} - \mu_0), \quad (\text{A.13})$$

$$\sigma^2(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}) - \mathbf{k}_x^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_x, \quad (\text{A.14})$$

where $\mathbf{K} = k(\mathbf{X}, \mathbf{X})$, $\mathbf{k}_x = k(\mathbf{X}, \mathbf{x})$, $\mu_0 = \mu_0(\mathbf{X})$, and σ_n^2 is the observation noise variance.

In AURORA, we use the posterior variance $\sigma^2(\mathbf{x})$ as a geometric reconstruction uncertainty metric. This uncertainty is high in regions less covered by the current observations and low near well-observed anchors.

C Details of NBV-to-Action Mapping

This section details how the selected NBV direction is mapped to a feasible in-hand rotation primitive. The NBV module outputs a desired viewing direction $\mathbf{u}_{\text{NBV}}^*$ in the object-centered planning frame. Since the low-level reorientation policy executes rotation-axis commands in the hand-centered world frame W , we first transform the NBV direction into W .

Let $T_{CO}(t) \in SE(3)$ denote the tracked object pose in the camera frame, and let $T_{WC} \in SE(3)$ denote the calibrated camera-to-world transform. The object pose in the hand-centered world frame is

$$T_{WO}(t) \triangleq T_{WC} T_{CO}(t), \quad (\text{A.15})$$

with rotation component $\mathbf{R}_{WO}(t)$. The desired viewing direction in W is then given by

$$\mathbf{d}_W = \frac{\mathbf{R}_{WO}(t) \mathbf{u}_{\text{NBV}}^*}{\|\mathbf{R}_{WO}(t) \mathbf{u}_{\text{NBV}}^*\|_2}. \quad (\text{A.16})$$

We define $\mathbf{v}_W$ as the unit vector from the current object center to the camera, expressed in W . We compute the minimal rotation that aligns $\mathbf{d}_W$ with $\mathbf{v}_W$. The rotation angle is

$$\theta = \arccos(\text{clip}(\mathbf{v}_W^\top \mathbf{d}_W, -1, 1)), \quad (\text{A.17})$$

where clipping improves numerical stability. When $\mathbf{v}_W$ and $\mathbf{d}_W$ are not parallel, the rotation axis is

$$\mathbf{k} = \frac{\mathbf{d}_W \times \mathbf{v}_W}{\|\mathbf{d}_W \times \mathbf{v}_W\|_2}. \quad (\text{A.18})$$

481 The corresponding minimal alignment rotation is

$$R_{\text{err}} = \exp(\theta[\mathbf{k}]_{\times}), \quad (\text{A.19})$$

482 where $[\mathbf{k}]_{\times}$ is the skew-symmetric matrix of $\mathbf{k}$.

483 We represent this rotation as a Lie algebra vector:

$$\boldsymbol{\omega} = \text{vee}(\log(R_{\text{err}})) = \theta \mathbf{k} \in \mathbb{R}^3, \quad (\text{A.20})$$

484 where $\text{vee}(\cdot)$ maps a skew-symmetric matrix to its vector representation. In the main paper, we use
485 $\log(R_{\text{err}})$ to denote this vectorized rotation representation for compactness.

486 Finally, we project the desired rotation vector onto the discrete action set supported by the low-level
487 in-hand rotation policy:

$$\mathbf{a}^* = \arg \max_{\mathbf{a} \in \mathcal{A}} \mathbf{a}^{\top} \boldsymbol{\omega}. \quad (\text{A.21})$$

488 This selects the feasible rotation primitive whose axis is most aligned with the desired viewpoint-
489 correcting rotation.

490 For numerical robustness, when $\|\mathbf{d}_W \times \mathbf{v}_W\|_2 < \epsilon$, we treat the two directions as nearly parallel. If
491 $\mathbf{v}_W^{\top} \mathbf{d}_W > 0$, we set $\boldsymbol{\omega} = \mathbf{0}$, indicating that no corrective rotation is required. If $\mathbf{v}_W^{\top} \mathbf{d}_W < 0$, we
492 choose an arbitrary unit vector orthogonal to $\mathbf{v}_W$ as $\mathbf{k}$ and set $\theta = \pi$. In practice, this degenerate
493 case rarely occurs because the NBV directions are discretely sampled and the action space is coarse.

494 D Details of 6D Pose Tracking and Keyframe Selection

495 This section provides additional details on the object pose tracking and keyframe selection strategy
496 used in Sec. 3.3. AURORA uses BundleTrack [18] to estimate the object pose in the camera frame,
497 $T_{CO}(t) \in SE(3)$, from segmented RGB-D observations. BundleTrack maintains a pool of masked
498 keyframes and jointly refines their poses through bundle adjustment. The resulting object poses al-
499 low depth observations from different viewpoints to be transformed into a consistent object-centered
500 frame for fusion.

501 However, during in-hand manipulation, hand-object occlusions and imperfect segmentation can
502 introduce unreliable frames. Directly inserting these frames into the keyframe pool may corrupt the
503 fused geometry and destabilize the pose graph. We therefore apply a lightweight keyframe filter
504 based on visibility, pose stability, and viewpoint change.

505 Let t^- denote the most recent selected keyframe. For a candidate frame at time t , we define the
506 relative motion with respect to t^- as

$$\Delta T(t) \triangleq T_{CO}(t^-)^{-1} T_{CO}(t), \quad (\text{A.22})$$

507 with rotation and translation components $\Delta \mathbf{R}(t)$ and $\Delta \mathbf{t}(t)$, respectively. The relative rotation and
508 translation magnitudes are

$$\Delta \theta(t) = \cos^{-1} \left(\frac{\text{tr}(\Delta \mathbf{R}(t)) - 1}{2} \right), \quad \Delta p(t) = \|\Delta \mathbf{t}(t)\|_2. \quad (\text{A.23})$$

509 We also compute the pose-update magnitude

$$e(t) = \|\log(\Delta T(t))\|_2, \quad (\text{A.24})$$

510 where $\log(\cdot)$ denotes the Lie algebra mapping from $SE(3)$ to its tangent space.

511 A frame is retained as a keyframe if it satisfies both reliability and informativeness criteria:

$$t \in \mathcal{K} \iff (A_t \geq \tau_A) \wedge (e(t) \leq \tau_E) \\ \wedge \max \left(\frac{\Delta \theta(t)}{\tau_R}, \frac{\Delta p(t)}{\tau_T} \right) \geq 1. \quad (\text{A.25})$$

512 Here, A_t denotes the visible object area measured from the segmentation mask $\mathbf{M}_t$. The threshold
513 τ_A rejects frames with insufficient visible object pixels, while τ_E filters out frames with unstable

pose updates. τ_R and τ_T are the rotation and translation thresholds, ensuring that the selected frame provides sufficient viewpoint change relative to the most recent keyframe, either through rotation or translation. This prevents redundant frames from being inserted while preserving informative observations for multi-view fusion.

In our implementation, this filtering step is applied before the online reconstruction update. Only the selected keyframes that pass the filter $\mathcal{K}$ are used to update the fused point cloud $\mathcal{P}_t$. This improves the stability of both point-cloud fusion and subsequent Ray-GPIS uncertainty estimation.

E Details of Mesh Extraction

AURORA represents the online reconstruction as a lightweight point cloud $\mathcal{P}_t$, enabling low-latency uncertainty estimation and active planning. After exploration terminates, we optionally run an offline post-processing stage to obtain a watertight mesh for quantitative evaluation and visualization.

To perform mesh reconstruction, given the final fused point cloud $\mathcal{P}_T$, we first estimate globally consistent surface normals using FaCE [39]:

$$\hat{\mathbf{N}} \triangleq \mathcal{F}(\mathcal{P}_T), \quad (\text{A.26})$$

where $\mathcal{F}(\cdot)$ denotes the FaCE-based normal estimation operator and $\hat{\mathbf{N}}$ is the estimated normal field. We use FaCE because the online fused point cloud may contain non-uniform density, positional noise, and residual misalignment from pose-tracking errors. More consistent normals improve the quality of the subsequent mesh reconstruction.

We then reconstruct a watertight mesh using NKSR [40]:

$$\mathcal{M} \triangleq \mathcal{R}_{\text{nksr}}(\mathcal{P}_T, \hat{\mathbf{N}}), \quad (\text{A.27})$$

where $\mathcal{R}_{\text{nksr}}(\cdot)$ denotes the NKSR reconstruction operator. The resulting mesh $\mathcal{M}$ is used only for final evaluation and qualitative visualization, and is not used by the online planner. During active exploration, Ray-GPIS operates directly on the online point cloud $\mathcal{P}_t$, avoiding the latency of repeated mesh extraction.

F Experimental Details

F.1 Planner and Reconstruction Hyperparameters

The main parameters used in our experiments are listed in Tab. A.1.

Table A.1: Planner and reconstruction hyperparameters.

Parameter	Value
Visibility threshold τ_A	0.25
Pose stability threshold τ_E	0.03
Rotation threshold τ_R	10°
Translation threshold τ_T	0.008 m
Ray hit threshold τ_{hit}	0.005 m
Angular similarity threshold γ	0.6
Angular interpolation bandwidth τ_{interp}	0.08
Novelty scale β	4.0
Maximum ray extent $s_{\text{max}} / t_{\text{max}}$	2.2
Radial band width $\Delta s / \Delta t$	0.01
Angular neighborhood δ_u	10°
Replanning interval	6 s

538

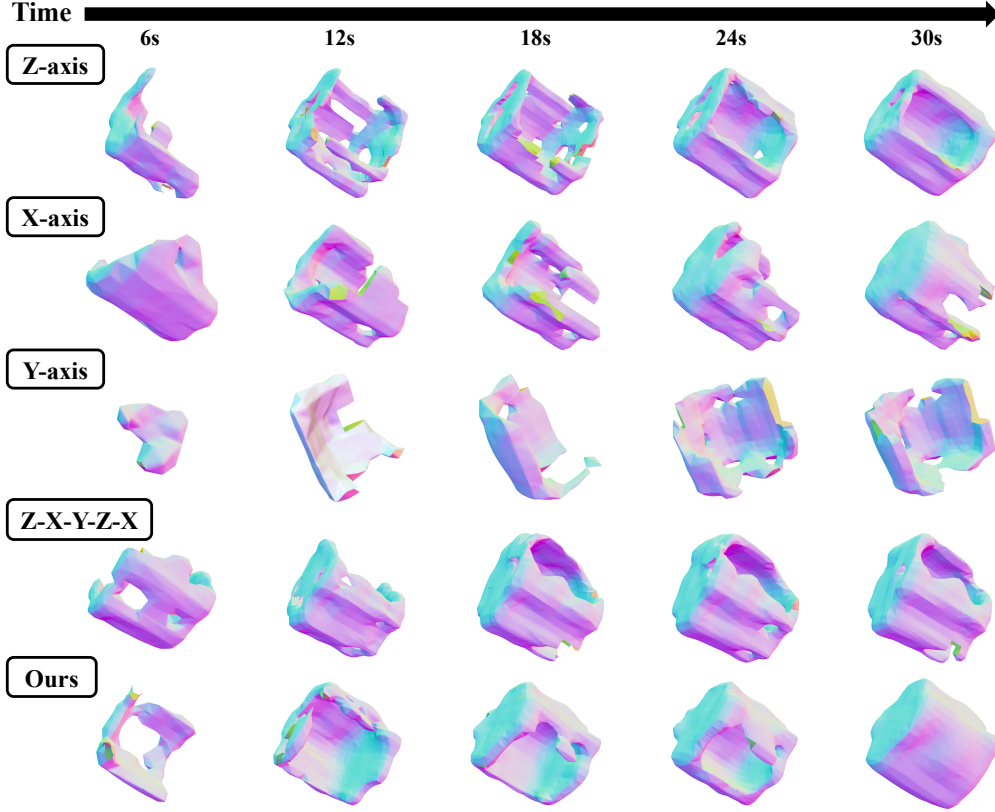


Figure A.3: Qualitative comparison of reconstruction results on *Cross Block* between AURORA and fixed-axis baselines.

539 F.2 Additional Comparison with Open-loop Rotation Baselines

540 Fig. A.3 provides additional qualitative comparisons between AURORA and the open-loop rotation
 541 baselines introduced in Sec 4.3. Single-axis rotation strategies i.e., the x -, y -, or z -axis, provide only
 542 a limited set of viewpoints. As a result, some surface regions remain persistently occluded by the
 543 hand or are never brought into view, leading to incomplete reconstructions and visible missing areas.
 544 The predefined Z-X-Y-Z-X rotation schedule exposes more diverse viewpoints, but it remains open-
 545 loop and cannot adapt to the current reconstruction state. Therefore, some fine geometric details
 546 remain insufficiently recovered, especially in regions that require targeted viewpoint changes. In
 547 contrast, AURORA actively selects the next rotation axis based on the estimated reconstruction
 548 uncertainty, allowing it to target under-observed and high-uncertainty regions and produce more
 549 complete surfaces over time.

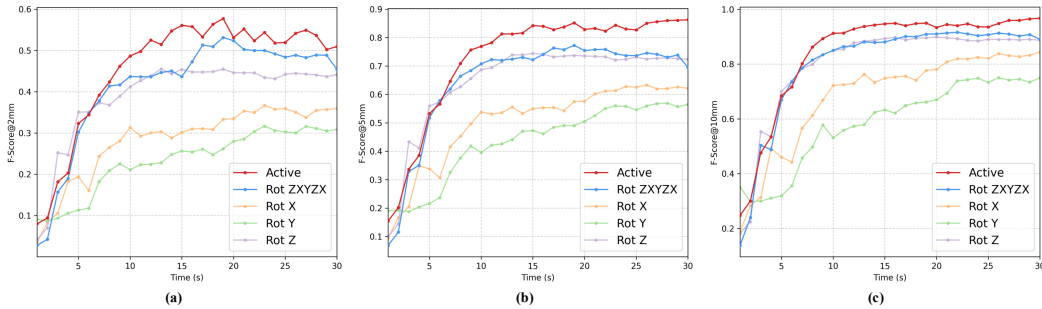


Figure A.4: Temporal evolution of Mesh-Mesh F-scores for AURORA and non-active baselines under different distance tolerances: (a) 2 mm, (b) 5 mm, and (c) 10 mm, averaged over six objects.

We further evaluate reconstruction efficiency using the temporal Mesh–Mesh F-score curves. At each timestamp, we reconstruct a mesh from the online point cloud fused up to that time using the same post-processing pipeline and compare it with the scanned ground-truth mesh. Fig. A.4 reports the averaged F-scores over six objects under three distance thresholds: 2 mm, 5 mm, and 10 mm.

Because AURORA always starts with a rotation about the z -axis; therefore, its F-score curve nearly overlaps with other baselines during the first 6 s. After 6 s, the open-loop baselines improve more slowly because repeated observations along the same or predefined axes provide diminishing information gain. Although switching axes according to a predefined schedule can improve reconstruction and achieves performance close to AURORA in some trials, it is less reliable overall. For instance, object slip during in-hand manipulation can change the actual object pose, preventing the predefined schedule from consistently exposing the intended unseen surfaces. In contrast, AURORA replans from the current reconstruction state and is therefore more robust on average, maintaining faster F-score improvement and reaching higher accuracy sooner.

The curves tend to saturate after approximately 18 s for two main reasons. First, for simple objects, most surfaces have already been observed by this time, leaving limited room for further F-score improvement. For more complex geometries, such as the *Cross Block* shown in Fig. A.3, AURORA can still fill missing mesh regions after 18 s. Second, in practice, the camera-to-object distance, sensor resolution, accumulated 6D tracking errors, and the downsampling and denoising operations used during point-cloud fusion all limit the mesh resolution and introduce small geometric deviations. Consequently, even our method indeed completes previously missing regions, the mesh extraction process can introduce slight distortions or misalignments elsewhere. Under fixed distance thresholds, these deviations can offset the F-score gain brought by the newly observed regions. Thus, this saturation does not indicate a failure of the active strategy.

We also note that the three single-axis baselines behave differently. This is caused not only by viewpoint coverage, but also by hardware constraints. Due to the sim-to-real gap in tactile feedback utility, contact dynamics, and friction, rotations around the x - and y -axes are less efficient and stable than the z -axis rotation and are more likely to induce object slip or unstable motions. Consequently, their reconstruction performance can be worse even under the same manipulation budget.

E.3 Comparison with Single-View 3D Reconstruction Baselines

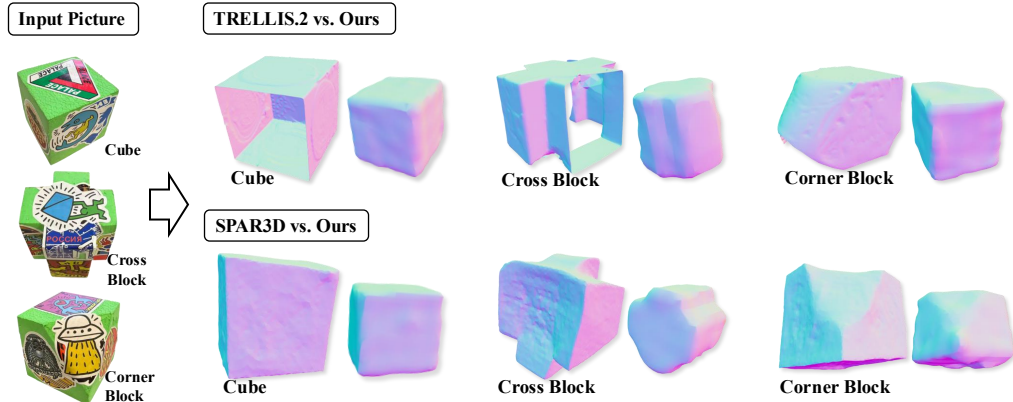


Figure A.5: Visual comparison of reconstruction failure modes for single-view baselines (TRELLIS.2 [41] and SPAR3D [42]) versus our method on the *Cube*, *Cross Block*, and *Corner Block*. These baseline reconstructions are generated from the high-quality input images shown on the left.

To further analyze the limitations of single-view 3D reconstruction under in-hand occlusion, we compare AURORA with two recent single-image baselines, TRELLIS.2 [41] and SPAR3D [42]. Although these methods are not designed for active in-hand reconstruction, they provide useful reference baselines because they also reconstruct 3D geometry from incomplete visual observations.

Table A.2: Mesh–Mesh reconstruction comparison with single-view 3D reconstruction baselines. $F@τ$ denotes the F-score under distance threshold $τ$, where higher is better.

Object	TRELLIS.2 [41]			SPAR3D [42]			Ours		
	$F@2$	$F@5$	$F@10$	$F@2$	$F@5$	$F@10$	$F@2$	$F@5$	$F@10$
Cube	0.8922	0.9324	0.9579	0.4510	0.5515	0.8968	0.6481	0.9557	0.9957
Corner Block	0.5140	0.6447	0.7833	0.4507	0.5331	0.8686	0.5298	0.8559	0.9488
L-shaped Block	0.8124	0.9251	0.9632	0.5209	0.7244	0.9987	0.4674	0.8367	0.9450
Pepper	0.6612	0.9504	0.9978	0.6998	0.9459	1.0000	0.5480	0.8890	0.9913
Cylinder	0.5884	0.8861	0.9423	0.9030	0.9963	1.0000	0.4035	0.8320	0.9372
Cross Block	0.4548	0.6509	0.7660	0.3176	0.6809	0.9260	0.4601	0.8050	0.9850
Mean	0.6538	0.8316	0.9018	0.5572	0.7387	0.9484	0.5095	0.8624	0.9672

We evaluate all methods using the Mesh–Mesh F-score at 2 mm, 5 mm, and 10 mm thresholds. The quantitative results are reported in Tab. A.2.

As shown in Tab. A.2, our method achieves higher mean Mesh–Mesh F-scores than TRELLIS.2 [41] and SPAR3D [42] at the 5 mm and 10 mm thresholds across all evaluated objects. This suggests that actively acquiring additional observations improves metric reconstruction over single-view priors. Nevertheless, single-view methods produce relatively clean meshes, highlighting a direction for us to make future improvements.

We observe the following failure modes for the two single-view baselines in Fig. A.5. TRELLIS.2 [41] preserves global structure and planar surfaces well, such as the rectangular faces of the *Cube* and *Cross Block*. However, it often fails to maintain topological completeness for objects with sharp edges or cutouts, producing open surfaces, incomplete cutting planes, and over-smoothed details. In contrast, SPAR3D [42] tends to generate more watertight meshes but with poorer geometric alignment. It often hallucinates rear-side geometry from the front view without enforcing global consistency, causing distortions such as non-orthogonal angles and warped surfaces, as seen in the *Cube* and *Corner Block* examples. These failures show that single-view methods struggle to infer coherent 3D geometry from limited observations, whereas incremental viewpoint acquisition is important for both completeness and geometric fidelity.

Both baselines also share three limitations in our in-hand setting. First, their performance depends strongly on input image quality. The results in Tab. A.2 use relatively clean, high-resolution object images without dexterous-hand occlusion, whereas realistic in-hand observations often suffer from partial visibility and motion blur. Second, their meshes do not preserve metric scale. We therefore rescale each generated mesh to the corresponding ground truth before computing Mesh–Mesh metrics. This scale ambiguity prevents direct use in metric-aware downstream robotic tasks. Third, they cannot recover true object texture from occluded or unobserved regions; any missing appearance must be hallucinated from learned priors. In contrast, AURORA actively reorients the object, incrementally fuses newly observed geometry in a consistent object-centric frame, and produces metrically consistent reconstructions.

F.4 Comparison with NeuralFeels & Discussions

NeuralFeels [3] is closely related to our work, as both methods reconstruct object geometry during in-hand manipulation. However, their task formulations differ, making a controlled direct comparison difficult. Specifically, NeuralFeels uses a perceptually open-loop in-hand rotation policy that focuses on rotating the object along a fixed axis and reconstructs geometry from both visual and tactile interaction sequences to compensate for occlusion. In contrast, our method focuses on closed-loop planning: it actively selects the next in-hand reorientation based on current reconstruction uncertainty to observe visually occluded regions, but does not use tactile sensing.

Due to differences in hardware, tactile sensing, and manipulation policies, we cannot directly reproduce NeuralFeels on our platform. We therefore conduct a limited comparison on the common

Table A.3: Limited comparison with NeuralFeels on the common *Pepper* object under a matched 30 s interaction budget. $F@7$ denotes the Mesh–Mesh F-score, where higher is better.

Method	$F@5$
NeuralFeels [3]	0.76
Ours	0.89

620 *Pepper* object, for which NeuralFeels provides released recordings. Both methods are evaluated under
 621 a matched 30 s interaction budget using the Mesh–Mesh F-score. Results are shown in Tab. A.3,
 622 with additional metric curves in Fig. A.6. The NeuralFeels results are reproduced using the authors’
 623 released code and data without customization; we plot corresponding F-scores for comparison.

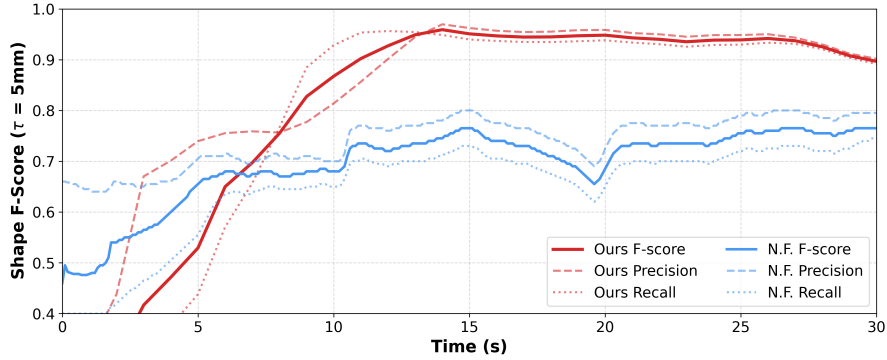


Figure A.6: Quality metrics on *Pepper*, comparing AURORA with NeuralFeels [3].

624 AURORA achieves an $F@5$ score of 0.89, while NeuralFeels achieves an average score of 0.76
 625 under the same 30 s interaction budget. This comparison is subject to differences in hardware, tactile
 626 sensing, manipulation policies, task focus, and object instances; therefore, the higher score should
 627 not be interpreted as evidence that AURORA is strictly superior to NeuralFeels. In particular, we
 628 acknowledge that NeuralFeels can track textureless objects, whereas our approach requires sufficient
 629 visual texture and may fail under weakly textured conditions. The observed performance gap may
 630 also be partly attributable to object appearance: our pepper object has a similar shape but richer
 631 texture than the one shown in the NeuralFeels video. Thus, this result does not necessarily imply
 632 superior reconstruction quality of our framework.

633 Nevertheless, this case study suggests two potential practical benefits of our design in this setting.
 634 First, active uncertainty-guided reorientation improves reconstruction efficiency (refer to the slope
 635 rate of curves): our planner explicitly selects actions that expose under-observed regions, whereas
 636 NeuralFeels does not use reconstruction uncertainty to choose the next in-hand action. In addition,
 637 our multi-axis in-palm reorientation provides more diverse viewpoints than the fingertips’ twisting
 638 motions used in NeuralFeels, which may leave the bottom surface and other occluded regions in-
 639 sufficiently observed. Second, our pipeline is more computationally efficient. For the same 30 s
 640 observation sequence, NeuralFeels requires 417.13 s of wall-clock reconstruction time, whereas
 641 our pipeline takes 71.82 s, including mesh extraction. This efficiency comes mainly from using a
 642 lightweight point-cloud representation for online fusion and planning instead of optimizing a dense
 643 neural SDF.